
Segmentación Semántica de Imágenes

Documentación Técnica

Sistema completo de Computer Vision sobre ADE20K (150 clases)

SegFormer-B0

PyTorch

HuggingFace

Gradio

albumentations

ADE20K

Python 3.10+

Gonzalo Solans del Carre Díaz-Gálvez · Mayo 2026

Índice de contenidos

- 1. Descripción general del proyecto
- 2. Stack tecnológico
- 3. Arquitectura del modelo — SegFormer-B0
- 4. Estructura de carpetas
- 5. Etapa 1 — Adquisición de datos
- 6. Etapa 2 — Preprocesado y aumentación
- 7. Etapa 3 — Entrenamiento
- 8. Etapa 4 — Inferencia
- 9. Etapa 5 — Aplicación Gradio
- 10. Tests automáticos
- 11. Guía de ejecución

1. Descripción general del proyecto

El proyecto construye un sistema completo de segmentación semántica que, dada cualquier imagen fotográfica, produce una versión coloreada donde cada categoría de objeto (cielo, coches, personas, edificios, vegetación, etc.) aparece pintada con un color distinto. El resultado es un mapa denso de etiquetas que asigna una clase a cada píxel de la imagen.

El sistema recorre todas las fases del pipeline de Computer Vision moderno: obtención automática de datos, preprocesado, fine-tuning de un modelo transformer preentrenado, inferencia sobre imágenes nuevas y despliegue como aplicación web interactiva.

Objetivo: demostrar el dominio del pipeline completo de Computer Vision aplicado a segmentación semántica, incluyendo el uso de datasets reales, modelos state-of-the-art y evaluación cuantitativa con métricas estándar (mIoU).

Etapa	Nombre	Descripción
1	Adquisición de datos	Descarga automática de ADE20K desde HuggingFace (3.700 imágenes)
2	Preprocesado	Normalización, aumentación con albumentations, DataLoaders
3	Entrenamiento	Fine-tuning de SegFormer-B0, 10 épocas, métricas mIoU
4	Inferencia	Segmentación de imágenes nuevas, overlay coloreado
5	Aplicación	Interfaz web Gradio con leyenda de clases y estadísticas

2. Stack tecnológico

El proyecto usa exclusivamente herramientas open-source del ecosistema Python para Computer Vision y Deep Learning.

Librería	Categoría	Uso en el proyecto
PyTorch ≥ 2.0	Framework Deep Learning	Red neuronal, entrenamiento, backpropagation, inferencia GPU/CPU
torchvision ≥ 0.15	Utilidades de visión	Transformaciones de tensor, utilidades de imagen
transformers ≥ 4.35	Modelos HuggingFace	Carga de SegFormerForSemanticSegmentation preentrenado
datasets ≥ 2.14	Datasets HuggingFace	Descarga y streaming de ADE20K (scene_parse_150)
albumentations ≥ 1.3	Aumentación	Pipeline de transformaciones: flip, brillo, rotación, blur, HSV
opencv-python ≥ 4.8	Procesado de imagen	Lectura/escritura de imágenes, operaciones morfológicas
Pillow ≥ 10.0	Manipulación	Carga de PNG/JPEG, conversión de modos de color
matplotlib ≥ 3.7	Visualización	Gráficas de entrenamiento, mapas de color, grid de resultados
Gradio ≥ 4.0	Interfaz web	App interactiva con subida de imagen, tabs, slider de opacidad
seaborn ≥ 0.12	Visualización estadística	Histogramas de distribución de clases

Librería	Categoría	Uso en el proyecto
numpy $\geq$ 1.24	Álgebra numérica	Operaciones sobre arrays de imágenes y máscaras
pandas $\geq$ 2.0	Tablas de datos	Estadísticas de clases en la app Gradio
pytest $\geq$ 7.4	Testing	Suite de tests automáticos por etapa

3. Arquitectura del modelo – SegFormer-B0

SegFormer es una arquitectura transformer para segmentación semántica propuesta por NVIDIA en 2021. Combina un encoder jerárquico Mix Transformer (MiT) con un decodificador MLP ligero, eliminando la necesidad de decodificadores complejos como los de U-Net o DeepLab.

Componentes principales

Componente	Descripción
Mix Transformer Encoder (MiT-B0)	Encoder jerárquico de 4 stages con Self-Attention de eficiencia lineal. Produce feature maps a escalas 1/4, 1/8, 1/16 y 1/32. La variante B0 tiene ~3,7 M parámetros.
Efficient Self-Attention	Reduce la complejidad cuadrática mediante un ratio de reducción de secuencia R: la dimensión de K y V se reduce por un factor configurable.
Mix-FFN	Feed-forward network con una convolución 3×3 de paso cero que introduce información posicional local sin embeddings explícitos de posición.

Componente	Descripción
All-MLP Decoder	Alinea los 4 stages a una dimensión común, hace upsample bilineal al tamaño 1/4, concatena, fusiona con una MLP y predice las 150 clases con una convolución 1×1.

Especificaciones técnicas

Parámetro	Valor
Variante usada	SegFormer-B0 (más eficiente, adecuada para hardware limitado)
Checkpoint base	nvidia/segformer-b0-finetuned-ade-512-512 (HuggingFace Hub)
Parámetros totales	~3,7 M (encoder) + ~0,4 M (decoder) = ~4,1 M parámetros
Resolución entrada	512 × 512 píxeles (RGB normalizado con stats ImageNet)
Resolución salida	128 × 128 logits → interpolación bilineal a 512 × 512
Clases de salida	150 clases semánticas (ADE20K scene_parse_150)
Pre-entrenamiento	ImageNet-1K + fine-tuning ADE20K por NVIDIA

SegFormer-B0 ofrece el balance adecuado para este proyecto: suficientemente preciso en ADE20K (~37 % mIoU sin fine-tuning propio) y lo bastante ligero para ejecutarse en CPU sin necesitar GPU dedicada. Además, HuggingFace ofrece el checkpoint ya fine-tuneado en ADE20K, lo que permite obtener resultados de calidad sin necesidad de reentrenar desde cero.

4. Estructura de carpetas

El proyecto sigue una organización modular donde cada directorio tiene un propósito único y las dependencias entre módulos son unidireccionales (la Etapa N solo importa de etapas anteriores).

semantic_segmentation/

```

├─ data/
│   ├─ raw/
│   │   ├─ train/          img_000000.jpg + mask_000000.png × 3.000
│   │   ├─ validation/    × 500 pares imagen/máscara
│   │   └─ test/          × 200 pares imagen/máscara
│   │   └─ metadata.json   Resumen del dataset (splits, clases, resolución)
│   └─ processed/
│       └─ dataset_stats.json Media/std por canal, píxeles/clase, pesos
├─ models/
│   └─ checkpoints/
│       ├─ best_model.pth   Mejor checkpoint (mayor mIoU en val)
│       ├─ last_model.pth   Checkpoint de la última época
│       └─ training_history.json
├─ outputs/
│   ├─ test_results/
│   └─ plots/
├─ src/
│   ├─ etapa1_datos.py
│   ├─ etapa2_preprocesado.py
│   ├─ etapa3_entrenamiento.py
│   ├─ etapa4_inferencia.py
│   └─ etapa5_app.py
├─ tests/
│   ├─ conftest.py
│   └─ test_etapa[1-4].py
├─ run_pipeline.py
└─ requirements.txt

```

5. Etapa 1 — Adquisición de datos

5.1 Dataset ADE20K (scene_parse_150)

ADE20K es uno de los benchmarks de referencia en segmentación semántica. La variante `scene_parse_150` contiene imágenes de escenas de interior y exterior con anotaciones píxel a píxel para 150 categorías.

Parámetro	Valor
Fuente	MIT CSAIL — distribuido vía HuggingFace Hub
Identificador HF	<code>scene_parse_150</code>
Total imágenes disponibles	~25.574 (train) + 2.000 (val)
Imágenes usadas	3.000 (train) + 500 (val) + 200 (test)
Clases	150 categorías semánticas
Formato imagen	JPEG RGB, resolución variable (~400-800 px)
Formato máscara	PNG uint8, valores enteros 0-149 (índice de clase)

5.2 Proceso de descarga y organización

La descarga es automática y reproducible mediante la API de HuggingFace Datasets, sin necesidad de descargar manualmente ningún archivo. Se llama a `load_dataset('scene_parse_150')`, se iteran los splits `train` y `validation` guardando pares imagen/máscara con nombres consecutivos, y se genera `metadata.json` con estadísticas del dataset.

Distribución de clases: ADE20K está muy desbalanceado. Las clases `'wall'`, `'building'`, `'sky'` y `'floor'` dominan en píxeles, mientras que `'microwave'`, `'flag'` o `'crt screen'` tienen muy pocos. Este desbalance se compensa en la Etapa 3 con una `CrossEntropyLoss` ponderada.

6. Etapa 2 — Preprocesado y aumentación

6.1 Pipeline de transformaciones

Se definen tres pipelines con albumentations, adaptados a cada split. La normalización sigue las estadísticas de ImageNet porque el encoder MiT-B0 fue preentrenado sobre ese dataset.

Pipeline	Transformaciones	Propósito
train_transform	Resize(512,512) → HorizontalFlip(p=0.5) → RandomBrightnessContrast(p=0.3) → RandomRotate90(p=0.2) → Normalize(ImageNet) → ToTensorV2	Aumentación aleatoria
val_transform	Resize(512,512) → Normalize(ImageNet) → ToTensorV2	Evaluación determinista
test_transform	Igual que val_transform	Comparar métricas

6.2 Aumentaciones principales

HorizontalFlip(p=0.5): volteo horizontal con probabilidad 50 %; es la aumentación más efectiva para escenas de interior/exterior.

RandomBrightnessContrast(p=0.3): altera brillo y contraste en un rango de ± 30 % para robustecer el modelo frente a distintas condiciones de iluminación.

Normalize (ImageNet): obligatorio para modelos preentrenados en ImageNet. Convierte píxeles de [0, 255] a floats normalizados alrededor de cero con desviación ~ 1 .

Nota sobre consistencia: todas las transformaciones se aplican de forma conjunta a imagen y máscara para preservar la correspondencia píxel a píxel.

6.3 SegmentationDataset y DataLoaders

Configuración	Valor
Carga de imagen	PIL.Image + conversión a NumPy RGB (H, W, 3)

Configuración	Valor
Carga de máscara	PIL.Image → NumPy uint8 → clip [0,149] → int64
Valores de retorno	Tensor imagen (3, 512, 512) float32 + Tensor máscara (512, 512) long
DataLoader train	batch=8, shuffle=True, num_workers=2, pin_memory=True, drop_last=True
DataLoader val/test	batch=4, shuffle=False, num_workers=2, pin_memory=True

7. Etapa 3 — Entrenamiento

7.1 Fine-tuning de SegFormer-B0

En lugar de entrenar desde cero, se aplica fine-tuning sobre el checkpoint `nvidia/segformer-b0-finetuned-ade-512-512`. Se usa `ignore_mismatched_sizes=True` para reemplazar la cabeza de clasificación si el número de clases difiere.

Parámetro	Valor
Modelo base	<code>nvidia/segformer-b0-finetuned-ade-512-512</code>
Épocas	10
Learning rate inicial	6e-5 (AdamW)
Weight decay	0.01
Scheduler	CosineAnnealingLR (T_max=10, eta_min=1e-6)
Gradient clipping	max_norm=1.0
Batch size train / val	8 / 4 imágenes por iteración

7.2 Función de pérdida y optimizador

CrossEntropyLoss ponderada: se pondera con los `class_weights` calculados en la Etapa 2 para compensar el desbalance de clases (inversamente proporcional a la frecuencia, capped a $\times 10$).

AdamW: variante de Adam con weight decay desacoplado, estándar para transformers. El weight decay actúa como regularización L2 directamente sobre los pesos.

CosineAnnealingLR: el learning rate decrece desde $6e-5$ hasta $1e-6$ siguiendo una curva coseno, evitando oscilaciones al final del entrenamiento y favoreciendo la convergencia en mínimos más planos.

7.3 Métricas: mIoU

mIoU (mean Intersection over Union) es la métrica estándar en segmentación semántica. Para cada clase c , $IoU(c) = TP / (TP + FP + FN)$. El mIoU es la media sobre todas las clases presentes. SegFormer-B0 preentrenado alcanza $\sim 37\%$ mIoU en ADE20K sin fine-tuning adicional.

Se guardan dos checkpoints en `models/checkpoints/`: `best_model.pth` (se sobrescribe cuando la mIoU de validación mejora) y `last_model.pth` (al final de cada época). Ambos almacenan `state_dict` del modelo y del optimizador, número de época, mIoU y loss.

8. Etapa 4 — Inferencia

8.1 Pipeline de inferencia

La función central es `segmentar_imagen(ruta, modelo, device)`. Acepta cualquier imagen JPG/PNG de cualquier resolución y devuelve tres arrays NumPy: imagen original, máscara coloreada y overlay blended.

Paso	Operación
1. Carga	<code>PIL.Image.open()</code> + conversión RGB $\rightarrow$ NumPy (H, W, 3)

Paso	Operación
2. Preproceso	Resize(512,512) + Normalize ImageNet + ToTensorV2 → tensor (1,3,512,512)
3. Inferencia	modelo(pixel_values=tensor) → logits (1, 150, 128, 128)
4. Upsampling	F.interpolate(logits, size=(H_orig, W_orig), mode='bilinear') → (1, 150, H, W)
5. Argmax	logits.argmax(dim=1) → máscara de índices (H, W) con valores [0,149]
6. Colorización	COLOR_ARRAY[pred_mask] → imagen RGB (H, W, 3) usando indexación vectorizada
7. Overlay	alpha * colored_mask + (1-alpha) * original → imagen blended

8.2 Mapa de colores y evaluación en test

El COLOR_MAP es una lista de 150 colores RGB construida concatenando colormaps de matplotlib (tab20, tab20b, tab20c, Set1, Set2, Set3, Paired, Accent) para maximizar la distinción visual entre clases adyacentes. La asignación color → clase es determinista y fija. Se usa indexación NumPy vectorizada para generar la imagen coloreada en una sola operación $O(H \times W)$.

Sobre el split de test (200 imágenes) se calcula el mIoU global, el mIoU por clase (150 valores), la velocidad de inferencia en imágenes/segundo y se guardan los primeros 20 overlays en outputs/test_results/.

9. Etapa 5 – Aplicación Gradio

La etapa final envuelve el sistema en una aplicación web interactiva con Gradio. Cualquier usuario puede subir una imagen y obtener la segmentación al instante, sin tocar el código.

Característica	Detalle
Subida de imagen	Acepta JPG y PNG por drag & drop, click o portapapeles
Pestaña Segmentación	Máscara coloreada pura (150 colores, uno por clase)
Pestaña Overlay	Imagen original + segmentación blended con opacidad ajustable
Slider de opacidad	Control deslizante 0-100 % para mezclar original con máscara
Tarjetas resumen	Clases detectadas, clase dominante, tiempo de inferencia
Leyenda de colores	Chips de color con nombre de clase y % de píxeles, orden descendente
Galería de ejemplos	6 imágenes precargadas de data/samples/ para demo rápida
Modelo lazy-load	El modelo se carga en memoria solo la primera vez que se hace una predicción
Fallback de pesos	Si no existe best_model.pth, usa los pesos preentrenados de HuggingFace

La leyenda dinámica muestra solo las clases presentes en la predicción actual, ordenadas de mayor a menor área. El color del texto de cada chip (negro o blanco) se elige automáticamente según la luminancia del fondo para maximizar legibilidad. Para lanzar la app: `python3 src/etapa5_app.py`. La app abre en <http://localhost:7860>.

10. Tests automáticos

El proyecto incluye una suite de tests con `pytest`, organizada por etapas. Los tests verifican tanto la corrección estructural (shapes, tipos, rangos de valores) como la corrección funcional (el modelo aprende, la inferencia cubre todos los píxeles).

Fichero	Test	Qué verifica
test_etapa1.py	test_minimum_train_images	≥ 2.500 imágenes en train
	test_image_mask_pairs_exist	Cada imagen tiene su máscara (100 muestras)
	test_mask_values_in_range	Máscaras en [0, 149]
	test_metadata_file_exists	metadata.json existe y tiene 150 clases
test_etapa2.py	test_train_batch_shape	Shape batch = (8, 3, 512, 512)
	test_images_normalized	Media ≈ 0 , std razonable tras normalización
	test_dataset_stats_file_exists	dataset_stats.json con 150 class weights
test_etapa3.py	test_best_model_checkpoint_exists	best_model.pth existe
	test_model_improved_miou	mIoU final > mIoU inicial
	test_forward_pass_no_error	Forward pass sin errores ni NaN
test_etapa4.py	test_masks_cover_all_pixels	100 % píxeles cubiertos
	test_sky_detected_in_landscape	Clase 'sky' presente en imagen de paisaje
	test_colored_mask_shape_matches_original	Máscara coloreada = mismas dimensiones que original

Para ejecutar todos los tests: `pytest tests/ -v --tb=short`. Para una etapa concreta: `pytest tests/test_etapa1.py -v`.

11. Guía de ejecución

Requisitos previos

Python 3.10 o superior ya que ha mi me ha dado problemas

Instalación de dependencias:

```
cd semantic_segmentation/  
pip3 install -r requirements.txt
```

Acción	Comando	Descripción
Pipeline completo	<code>python3 run_pipeline.py</code>	Ejecuta etapas 1→5 en orden. ~3-4 h en CPU, ~45 min con GPU.
Solo la app	<code>python3 src/etapa5_app.py</code>	Lanza directamente la app con pesos preentrenados. ~2 min.
Saltarse entrenamiento	<code>python3 run_pipeline.py --skip-train</code>	Descarga datos + preprocesa + infiere con checkpoint existente.
Saltarse descarga	<code>python3 run_pipeline.py --skip-download</code>	Solo preprocesa, entrena e infiere.
Tests	<code>pytest tests/ -v</code>	Ejecuta la suite completa. Requiere las etapas previas completadas.

El modo más rápido para probar el sistema es instalar los requisitos y ejecutar `python3 src/etapa5_app.py`. El modelo se descarga automáticamente de HuggingFace (~90 MB) y la app abre en el navegador en <http://localhost:7860>. Se puede subir cualquier imagen y obtener la segmentación en pocos segundos.